



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251225
Nama Lengkap	RIFKI WAYU SHABAN RIZALDI
Minggu ke / Materi	07 / Percabangan dan Perulangan Kompleks

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA

YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

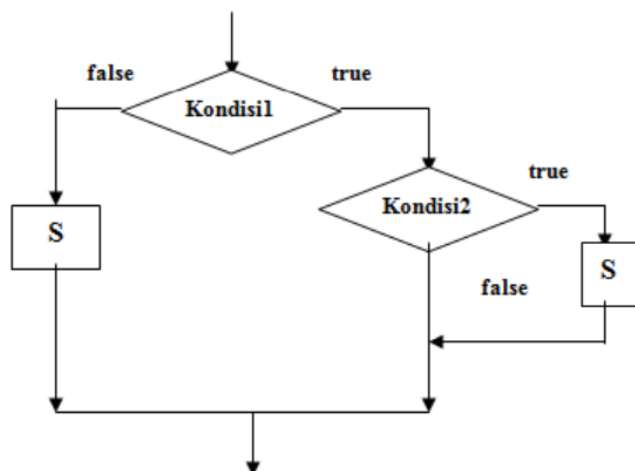
Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 1 Struktur Percabangan Kompleks

Percabangan kompleks bentuk 1 adalah struktur pengambilan keputusan di mana kondisi yang digunakan tidak hanya satu, melainkan terdiri dari banyak pilihan atau alternatif. Selain itu, perintah atau aksi yang dijalankan pada setiap kondisi juga bisa lebih dari satu, sehingga memungkinkan penanganan kasus yang lebih beragam dan kompleks:

```
1  if kondisi1:
2      if kondisi2:
3          S
4          S
5  else:
6      S
7      S
```

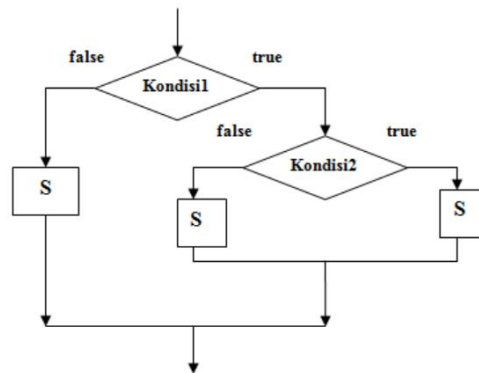
Berikut lampiran gambar flowcart di atas



Percabangan bentuk 2 :

```
1  if kondisi1:
2      if kondisi2:
3          S
4          S
5      else:
6          S
7          S
8  else:
9      S
10     S
```

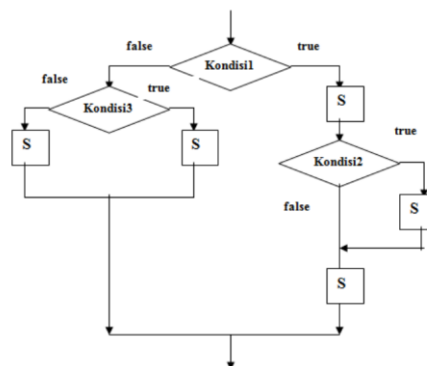
Bentuk gambar 2 flowcart



Percabangan kompleks bentuk 3:

```
1  if kondisi1:
2      S
3      if kondisi2:
4          S
5          S
6  else:
7      if kondisi3:
```

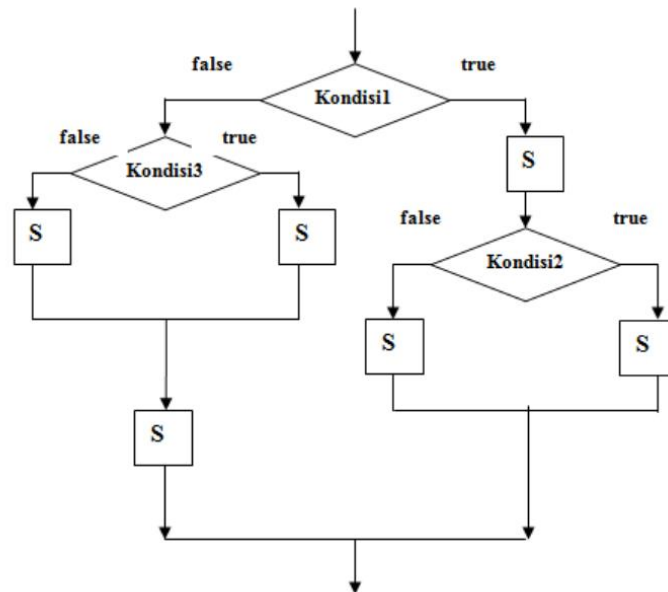
Berikut gambar flowchart pada bentuk percabangan kompleks



Percabangan kompleks bentuk 4

```
1  if kondisi1:
2      S
3      if kondisi2
4          S
5          S
6      else:
7          S
8          S
9  else:
10     if kondisi3
11         S
12         S
13     else:
14         S
15         S
16 S
```

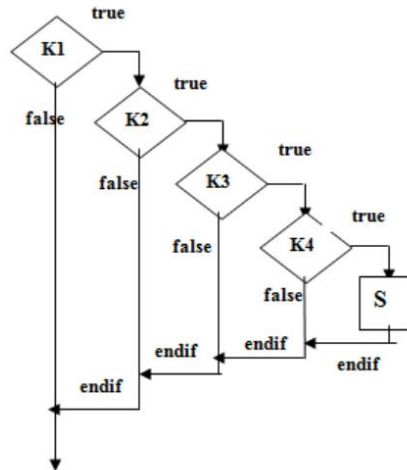
Berikut bentuk gambar flowchart 4



Percabangan kompleks bentuk 5

```
1  if kondisi1:
2      if kondisi2:
3          if kondisi3:
4              if kondisi4:
5                  S
```

Bentuk gambar flowchart dari percabangan kompleks bentuk 5



Percabangan kompleks bentuk 6

```
1  if kondisi1:
2      S
3  else:
4      if kondisi2:
5          S
6      else:
7          if kondisi3:
8              S
9          else:
10             if kondisi4:
11                 S
12             else:
13                 S
```

Pada kasus tertentu IF bertingkat memiliki keuntungan tersendiri. Dengan menggunakan IF bertingkat maka eksekusi perintah menjadi lebih baik dan efisien. Dengan menggunakan IF bertingkat maka tidak semua kondisi IF dikerjakan sehingga waktu eksekusi lebih cepat. Sedangkan pada IF biasa semua kondisi IF pasti akan dicoba satu per satu walaupun mungkin pada akhirnya hanya satu saja perintah IF yang terpenuhi. Hal ini tentu memperlambat proses. Contoh kasus yang cocok untuk IF bertingkat adalah kasus konversi nilai angka menjadi nilai huruf dengan batasan-batasan nilai yang sudah ditentukan sebelumnya.

```

1  if (kondisi1):
2      instruksi1
3  elif(kondisi2):
4      instruksi2
5  elif(kondisi3):
6      instruksi3
7  elif(kondisi4):
8      instruksi4

```

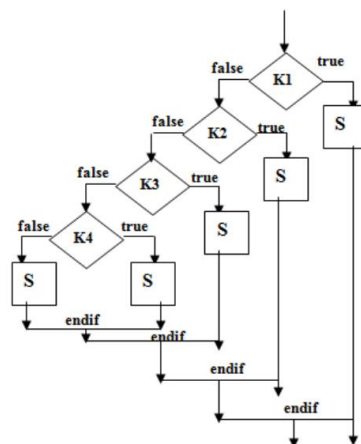
Bedakan dengan:

```

1  if (kondisi1):
2      instruksi1
3  if(kondisi2):
4      instruksi2
5  if(kondisi3):
6      instruksi3
7  if(kondisi4):
8      instruksi4

```

Bentuk flowchart bentuk 6



MATERI 2 Struktur Perulangan Kompleks Break

Perintah ini digunakan untuk menghentikan proses perulangan yang sedang terjadi. Biasanya disebabkan oleh suatu kondisi tertentu yang diimple-mentasikan menggunakan perintah IF.

```
1 for i in range(1000):
2     print(i)
3     if i==10:
4         break
```

Output:

```
0
1
2
3
4
5
6
7
8
9
10
```

Program di atas akan menampilkan angka 1 sampai dengan 10 walaupun di perulangan sudah diset dari 1 sampai dengan 1000. Hal ini karena perintah break yang diberikan pada saat kondisi i=10. Angka 10 masih ditampilkan karena perintah untuk mencetak diletakkan sebelum perintah break. Gambar break dari kode program demo break ada pada 6.7:

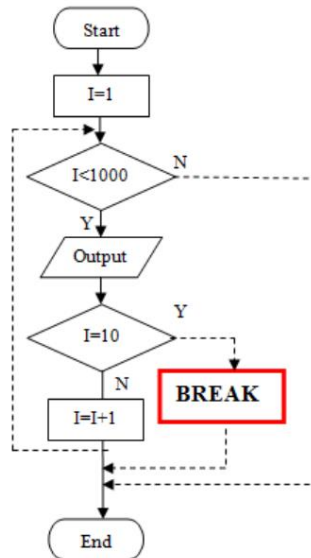
Contoh program break lain:

```
1     for i in range(1000):
2         if i==10:
3             break
4         print(i)
```

Output:

```
0
1
2
3
4
5
6
7
8
9
```

Program di atas akan menampilkan angka 1 sampai dengan 9 walaupun di perulangan sudah diset dari 1 sampai dengan 1000. Hal ini karena perintah break yang diberikan pada saat kondisi i=10. Angka 10 tidak ditampilkan karena perintah untuk mencetak diletakkan sesudah perintah break. Gambar break dari kode program demo break ada pada 6.8:



Berikut gambar flowchart menggunakan program break.

Continue

Perintah continue digunakan untuk menghentikan sisa proses dalam satu iterasi perulangan, lalu langsung melanjutkan ke iterasi berikutnya dari awal loop. Dengan kata lain, semua pernyataan yang berada setelah continue dalam satu putaran akan dilewati. Umumnya, penggunaan continue dikombinasikan dengan kondisi if untuk menentukan kapan perintah tersebut dijalankan. Seperti dibawah ini:

Input:

```

1  for i in range(10):
2      if i==5:
3          continue
4      print(i)

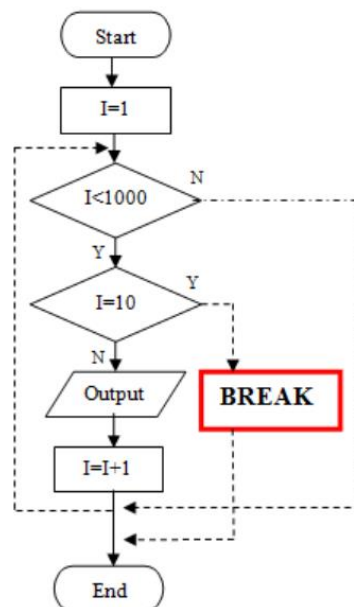
```

Output:

Output:

0
1
2
3
4
6
7
8
9

Program di atas tidak menampilkan angka 5 karena pada saat angka 5 akan ditampilkan, perintah



Beikut gambar flowchart yang menggunakan program break 2

Ketika continue dijalankan, perintah pencetakan yang berada setelahnya tidak akan dieksekusi, dan program akan langsung melanjutkan ke iterasi berikutnya dalam perulangan.

Perulangan Bertingkat

Struktur perulangan kompleks adalah jenis perulangan di mana terdapat perulangan lain di dalamnya, sehingga membentuk perulangan bertingkat. Kondisi ini membuat waktu

pemrosesan menjadi lebih lama karena adanya beberapa lapisan iterasi. Banyak algoritma tertentu yang memang cocok menggunakan struktur seperti ini.

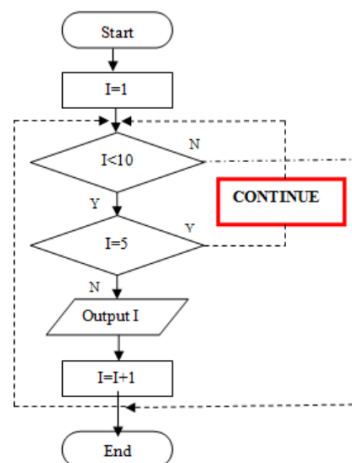
Beberapa contoh permasalahan yang dapat diselesaikan dengan perulangan kompleks antara lain pengolahan matriks dengan array dua dimensi, permainan berbasis papan seperti catur dan minesweeper, serta pengolahan citra digital, misalnya untuk mendeteksi tepi gambar atau mengubah gambar berwarna menjadi grayscale. Secara umum, permasalahan yang menggunakan perulangan kompleks biasanya memiliki pola berbentuk grid atau kotak-kotak dengan dimensi panjang dan lebar. Berikut adalah contoh perulangan kompleks:

```
1 for i in range(m):  
2     <lakukan perintah ini>  
3     <lakukan perintah itu>
```

Program 2:

```
1 for j in range(n):
```

Berikut gambar flowchart menggunakan program continue



```
2     <lakukan perintah ini>  
3     <lakukan perintah itu>
```

Program pertama melakukan perulangan sebanyak 10 kali, sementara program kedua melakukan perulangan sebanyak 5 kali. Kedua program tersebut berjalan secara terpisah. Namun, jika perulangan pada program kedua dimasukkan ke dalam perulangan program pertama, maka hasilnya menjadi:

```

1  for i in range(m):
2      for j in range(n):
3          <lakukan perintah ini di inner>
4          <lakukan perintah itu di inner>
5      <lakukan perintah lain di outer>
6      <lakukan perintah lain lagi di outer>

```

Bagian for i menjadi outer loop, sedangkan bagian for j menjadi inner loop. Alur pengerjaannya

adalah sebagai berikut: untuk setiap 1x s/d m outer loop dijalankan akan dikerjakan n kali inner loop.

Untuk sintaks dalam bentuk sama saja. Perhatikan contoh berikut:

```

1  while(i<=m):
2      while(j<=n):
3          <lakukan perintah ini di inner>
4          <lakukan perintah itu di inner>
5      <lakukan perintah lain di outer>
6      <lakukan perintah lain lagi di outer>

```

Bagian while i berfungsi sebagai perulangan luar (outer loop), sedangkan while j menjadi perulangan dalam (inner loop). Mekanisme kerjanya adalah setiap satu kali perulangan luar berjalan (dari 1 sampai m), maka perulangan dalam akan dieksekusi sebanyak n kali.

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github: <https://github.com/rifkiwayushabanR/week1>

SOAL 1

```

:  n = int(input("Silahkan masukan angka yang anda mau\n>> "))
    a = n
    ulg = True

    while ulg == True:
        if a % 2 == 0 and a != 2:
            a -= 1
        elif a % 3 == 0 and a != 3:
            a -= 1
        elif a % 5 == 0 and a != 5:
            a -= 1
        elif a % 7 == 0 and a != 7:
            a -= 1
        else:
            print(f"{a} adalah prima terdekat dari {n}")
            ulg = False

```

Hasil output:

Silahkan masukan angka yang anda mau	Silahkan masukan angka yang anda mau
>> 12	>> 21
11 adalah prima terdekat dari 12	19 adalah prima terdekat dari 21

Penjelasan:

- Penggunaan percabangan di dalam perulangan memungkinkan kita mencari angka prima terdekat. Program menggunakan while loop karena kita tidak tahu berapa langkah yang dibutuhkan untuk menemukan bilangan prima terdekat.
- Di dalam perulangan tersebut terdapat beberapa percabangan (if-elif) yang mengecek apakah angka habis dibagi bilangan tertentu (2, 3, 5, 7). Jika angka tersebut bukan prima, maka nilai akan dikurangi satu ($a -= 1$) dan dicek kembali.
- Proses ini terus berulang hingga ditemukan bilangan prima, kemudian program akan menampilkan hasilnya dan menghentikan perulangan.

SOAL 2

Code:

```
n = int(input("Silahkan masukan angka yang angka yang anda mau\n>> "))

a = 1
print()

for i in range(1, n+1):
    a = a * i

for i in range(n, 0, -1):
    c = []
    for j in range(i, 0, -1):
        c.append(str(j))
    print(int(a), " ".join(c))
    a = a / i

print()
```

Hasil output:

```
Silahkan masukan angka yang angka yang anda mau
>> 6

720 6 5 4 3 2 1
120 5 4 3 2 1
24 4 3 2 1
6 3 2 1
2 2 1
1 1
```

Penjelasan :

Melalui perulangan dalam perulangan, kita dapat membuat program faktorial seperti di atas. Hal ini menggunakan fungsi perulangan for di dalam fungsi for karena kita tahu kapan

program harus berhenti berdasarkan inputan dari pengguna. Perulangan juga terjadi dalam proses perhitungan faktorial seperti contoh di atas.

SOAL 3

Code:

```
tinggi = int(input("Silahkan masukan Tinggi\n>> "))
lebar = int(input("\nSilahkan masukan lebar\n>> "))
print()

a = 1
for i in range(1, tinggi+1):
    for j in range(a, lebar+a):
        print(j, " ", end='')
        a = j + 1
    print()
```

Hasil output:

```
Silahkan masukan Tinggi
>> 5

Silahkan masukan lebar
>> 4

1  2  3  4
5  6  7  8
9  10 11 12
13 14 15 16
17 18 19 20
```

Penjelasan:

Program ini menggunakan perulangan untuk mengatur penulisan angka sesuai dengan nilai lebar dan tinggi yang dimasukkan. Perulangan untuk lebar berada di dalam perulangan untuk tinggi, sehingga jumlah angka yang ditampilkan adalah hasil dari lebar dikali tinggi. Dengan cara ini, angka-angka tersebut akan membentuk pola seperti persegi atau persegi panjang.